TWIN-64
Architecture Document

Helmut Fieres
August 3, 2025

Contents

Introduction

Designers of the seventies and early eighties CPUs almost all used a micro-coded approach with hundreds of instructions. RISC was just starting to enter the stage. Registers were typically not really generic but often had a special function or meaning and many instructions were rather complicated and the designers felt they were useful. The compiler writers often used a small subset ignoring these complex instructions because they were so specialized. The later eighties and nineties shifted to RISC based designs with large sets of registers, fixed word instruction length, a large virtual memory address range and instructions that are in general much more pipeline friendly. Today, processors are highly complex with super-scalar deep pipelines and a multilevel cache hierarchy, just to name a few of the hallmarks of modern CPU design. What if there would be a simple design that one person can completely understand ?

Welcome to Twin-64. Twin-64 is a simple 64-bit CPU with a register-memory model and segmented virtual memory. The design is heavily influenced by Hewlett Packard's PA_RISC architecture, which was initially a 32 bit RISC-style load-store machine. Almost all of the key architecture features come from there. However, other processors, such as the Data General MV8000, the DEC Alpha, and the IBM PowerPc, were also influential or at least investigated for their architectural design choices. In addition the Blitz-64 architecture was investigated. Blitz-64, a design developed at Portland State University, offered an interesting approach in that it only support 64-bit signed arithmetic, avoiding common errors from mixing signed and unsigned arithmetic.

While it is not a design goal to build an industry strength, competitive CPU, the CPU should nevertheless be useful and largely follow established common design practices. For example, instructions should not be invented because they seem to be useful, but rather designed with the assembler and compilers in mind. Instruction set design is also CPU design. Register memory architectures were typically micro-coded complex instruction machines. In contrast, Twin-64 instructions will be hard coded and are in general "pipeline friendly", avoiding data and control hazards and stalls where possible.

Where does the name Twin-64 originate from? Obviously the "64" indicates that the CPU is a 64-bit CPU. The "Twin" part will become clearer when implementations of the CPU are described in later chapters. One implementation represents a dual-issue instruction design with two ALUs for computation. Nevertheless, a very basic implementation could just offer a single instruction issues, single ALU design as well.

This document describes the overall architecture, instruction set and runtime model for Twin-64. It is structured into several parts. The first part will give an overview on the system organization and architecture.The major part of the document then presents the instruction set. Memory reference instructions, immediate instructions, branch instructions, computational instructions and system control instructions are described in detail. These chapters are the authoritative reference of the instruction set. The next part will present the major components such as the processor itself, the I/O module architecture and finally the runtime model and calling conventions. An appendix will provide additional details on design and complementary notes.

System Organization

This section will introduce Twin-64 by highlighting the major components and design decision taken.

A Register Memory Architecture

Modern RISC CPUs are typically load/store CPUs and feature a large number of general registers. Operations takes place between registers. Twin-64 follows a slightly different route. There is a smaller number of general registers and in addition to computation between registers, a register-memory model is also supported. There are however no instructions that read and write to memory in one instruction cycle as this would complicate the design considerably.

Twin-64 implements a **register memory architecture** offering few addressing modes for the operand combined with a fixed instruction word length. For most instructions one operand is the register, the other is a flexible operand which could be an immediate, a register content or a memory address from where the data is obtained or placed. The result is placed in the register which is also the first operand. These type of machines are also called two address machines.

In contrast to similar historical register-memory designs, there is no operation which does a read and write operation to memory in the same instruction. For example, there is no "increment memory" instruction, since this would require two memory operations and is in general not pipeline friendly. Memory data access is performed on a machine word, half-word or byte basis. Besides the implicit operand fetch in a computational instruction, there are dedicated memory load / store instructions. In addition to the register-immediate operand and register memory-operand mode, one operand mode supports the three operand model (Rs = Ra OP Rb), specifying two registers and a result register, which allows to perform three address register for computational operations as well. The machine can therefore operate in a memory register model as well as a load / store register model.

Processor

The Twin-64 processor is only one element of a complete system. A system also includes memory and I/O modules. adapters, and interconnecting busses. The Central Processing Unit (CPU) includes a general register set and machine state registers. To support virtual memory addressing, a hardware translation lookaside buffer (TLB) is included on processors to provide virtual to physical address translations. A cache is optional, but for performance reasons a key component of the processor as well. A processor will always emit virtual addresses that need to be translated into physical addresses by the TLB.

Virtual Memory

Twin-64 offers a large global address range, organized in segments. Segment number and offset build the global virtual address. Segments are also associated with access rights and protection identifies. The CPU itself can run in user and privilege mode. Twin-64 emits at the CPU level virtual addresses. The global virtual memory range is a 52-bit space organized into a 20-bit segment Id and a 32-bit byte offset. There are 2^{20} segments with 2^{32} bytes.

Virtual addresses are translated to physical addresses by the TLB when memory is referenced. Address translations are made at the page level. For memory management reasons, the virtual address range can also be seen as a set of pages. A segment has thus 2^{18} pages of 16Kb size. A virtual page number combining the segment and page number in that segment is a 2^{38} number.

The first segments in the virtual address space are special in that they also represent the physical address range. A virtual address in that range bypasses the TLB and accesses physical memory directly.

For performance reasons, the TLB will support different pages sizes in addition to the architected 16Kbyte page size. There are 16Byte, 256Kbyte, 4Mbyte and 64Mbyte pages. It is very common for the operating system and data to be mapped in consecutive physical pages, which are then covered by a larger page size.

Security and Protection Support

Twin-64 implements a protection model along two dimensions. The first dimension is the **access control** and privilege level check. Each page is associated with a page type. There are read-only, read-write, execute and gateway pages. There are two privilege levels, user and supervisor mode. Access type and privilege level form the access control information field, which is checked for each instruction and data memory access.

The second dimension of protection is a **protection Id**. Twin-64 allows to record a set of segment Id values in the processor control registers. For each access to a segment that has the protection check bit set, one of the protection Id control registers must match the segment Id. If not, a protection violation trap is raised.

Protection Id checking is typically used to form a grouping of access. A good example is the stack data segment, which is accessible at user privilege level. Since the CPU features a global virtual memory address space, every task could access a data stack of another task. With protection Id checking enabled and the segment Id loaded in one of the control registers, only the corresponding user task is allowed to access the stack data segment with a matching protection Id.

Privilege Changes

Modern processors support the distinction between several privilege levels. Instructions can only access data or execute instructions when the privilege level matches the privilege level required. Two or four levels are the most common implementation. Changing between levels is often done with a kind of trap operation to a higher privilege level software, for example the operating system kernel. However, traps are expensive, as they cause a CPU pipeline to be flush when going through a trap handler.

Twin-64 implements gateways for privilege promotion. There are two privilege levels, **user** and **priv** mode. A special option on the unconditional branch instruction will raise the privilege level when the instruction is executed on a gateway page to the level of that gateway page. Return from a higher privilege level to a code page with lower privilege level, will automatically set the lower privilege level. A branch to a higher privilege level page that is not a gateway page will result in a privilege violation trap.

Each instruction fetch compares the privilege level of the page where the instruction is fetched from with the status register "P" bit. A higher privilege level on an execution page results in a privilege violation trap. A lower level of the execute page and a higher level in the status register will in an automatic demotion of the privilege level.

Debugging Support

A fundamental requirement is the ability to set instruction and data breakpoints. An instruction breakpoint is encountering the BRK instruction in an instruction stream. The break instruction will cause an instruction break trap and pass control to the trap handler. Just like the other traps, the pipeline will be emptied by completing the instructions in flight and flushing all instructions after the break instruction. Since break instructions are normally not in the instruction stream, they need to be set dynamically in place of the instruction normally found at this instruction address. The key mechanism for a debugger is then to exchange the instruction where to set a break point, hit the breakpoint and replace again the break point instruction with the original instruction when the breakpoint is encountered. Upon continuation and a still valid breakpoint for that address, the break point needs to be set after the execution of the original instruction. To facilitate this mechanism, a bit in the status word will provide an easy way to trap again after the instruction was executed.

Data breakpoints are traps that are taken when a data reference to the page is done. They do not modify the instruction stream. Depending on the data access, the trap could happen before or after the instruction that accesses the data. A write operation is trapped before the data is written, a read instruction is trapped after the instruction took place.

Input / Output

Twin-64 features a memory-mapped I/O architecture. A portion of the physical address space is dedicated to the I/O subsystem. Within this address range, I/O modules react to memory load and store instructions. Processors invoke I/O operations by executing load and store instructions to either virtual or absolute addresses. When implementing a driver, the standard page-level protection mechanism is applied during address translation. A driver, typically implemented running in privileged mode, can also grant a user program direct control by mapping an I/O page into virtual space without compromising system integrity.

Interrupt System

Interrupts and exceptions are events that occur asynchronously to the instruction execution. Depending on the type, they occur during the execution of an instruction or are checked in between instructions. Example for the former is the arithmetic overflow tap, example for the latter is an external interrupt. Depending on the interrupt or exception type, the instruction is restarted or execution continues with the next instruction.

Twin-64 implements a simple interrupt system. Each processor features an interrupt input register and an interrupt mask. When an I/O module receives a command, it also receives a bit mask and target module Id for sending the interrupt upon completion of the request. The interrupt data is compared to the interrupt mask register. If the particular bit is enabled and interrupts are in general enabled, the target processor will enter the interrupt handler.

Since causing an interrupt is actually just storing the interrupt request into the processor's interrupt register, I/O modules as well as processors themselves can raise an interrupt at the target processor.

Instruction Set Overview

Twin-64 features a simple and efficient instruction set. All instructions are of fixed length, which is a 32-bit word for Twin-64. There will be few instructions in total, however, depending on the instruction several options for the instruction execution are encoded to make an instruction more versatile. Great emphasis is placed in that such options do not overly complicated the pipeline and only increase the overall data path length slightly. Instead of a multitude of addressing modes, typically found in the CISC style CPUs, Twin-64 offers very few addressing modes with a simple base address - offset calculation model. No indirection or any addressing mode that would require to read a memory item for address calculation is part of the architecture.

The instruction set is divided into five general groups of instructions. The **memory reference** group contains the instructions to load and store to virtual and physical memory. The **immediate** group contains the instructions for building an immediate value up to 32 bit. The **branch** group present the conditional and unconditional instructions. The **computational** instructions perform arithmetic, boolean and bit functions such as bit extract and deposit. Finally, the **control** instruction group contains all instructions for managing HW elements such as caches and TLBs, register movements, as well as traps and interrupt handling.

Computational instructions

The arithmetic, logical and bit field operations instruction represent the computation type instructions. Twin-64 offers 64-bit signed arithmetic operation only.

The logical instructions allow to negate one operand as well as the output. The bit field instructions will perform bit extraction and deposit as well as double register shifting. These instruction not only implement bit field manipulation, they are also the base for all shift and rotate operations.

Immediate instructions

Several instructions have within the instruction word bit fields for representing immediate values. Nevertheless, fixed length instruction word size architectures have one issue in that there is not enough room to embed a full word immediate value in the instruction. Typically, a combination of two or three instructions that concatenate the value from parts is used.

Memory reference instructions

Memory reference instruction operate on memory and a general register with a unit of transfer being a word, a half-word or a byte. In that sense the architecture is a typical load/store architecture. However, in contrast to a load/store architecture Twin-64 load / store instructions

are not the only instructions that access memory. Being a register/memory architecture, all instructions with an operand field encoding may access memory as well. There are however no instructions that will access data memory access for reading and writing back an operand in the same instruction. Due to the operand instruction format and the requirement to offer a fixed length instruction word, the offset for an address operation is limited but still covers a large address range.

With the exception of the load reserved and store conditional instruction, memory reference instructions support a base register modification. When the option is set, the base register for an address computation will be adjusted before or after the data access. A negative offset will be added before the address computation, a positive offset after the address computation.

Branch instructions

The control flow instructions are divided into unconditional and conditional branch type instructions. The unconditional branch type instructions allow in addition to altering the control flow the saving of the return point to a general register. Upon subroutine return, there is a branch instruction using this value to return via a general register content.

Branch instructions are further grouped into instruction address relative and global virtual address relative. The former computes an offset in the current code segment, the latter in the specified target segment. In any case, branch targets are global virtual addresses.

System control instructions

The system control instructions are intended for the runtime designer to control the CPU resources such as TLB, Cache and so on. There are instruction to load a segment or control registers as well as instructions to store them. The TLB and the cache modules are controlled by instructions to insert and remove data in these two entities. Finally, the interrupt and exception system needs a place to store the address and instruction that was interrupted as well as a set of shadow registers to start processing an interrupt or exception. Most of the instructions found in this group require that the processor runs in privileged mode.

Processing Resources

General Registers

Twin-64 features a set of 16 general purposes registers. They are used for data and address values.

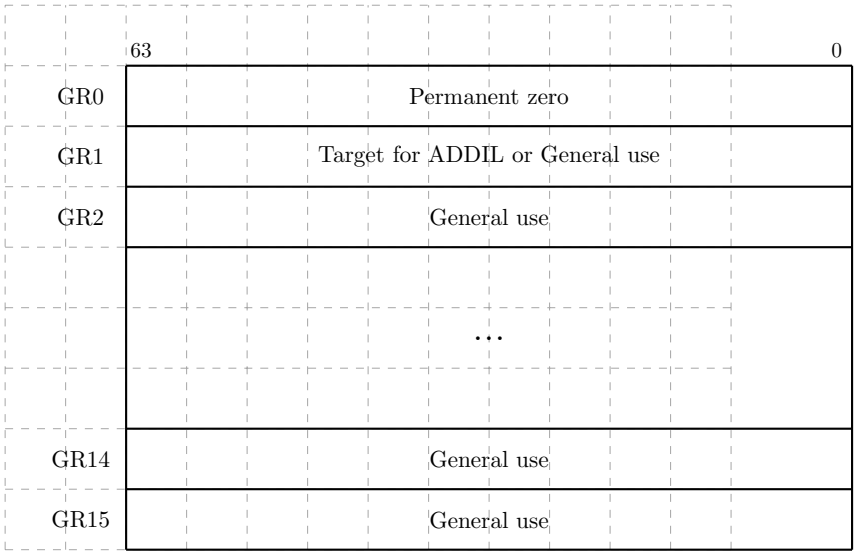


Figure 1: General registers

Some general registers have a dedicated use. Register zero will always return a zero value when read and a write operation will have no effect. Although there are only 16 general registers in the CPU, implementing such a register greatly simplifies the instruction design, in that it for example provides a target when the result is not needed. General register one is a scratch register and also serves as an implicit target for some instructions. Other general registers may have dedicated purpose defined in the runtime architecture.

Control Registers

Twin-64 has 16 control registers.

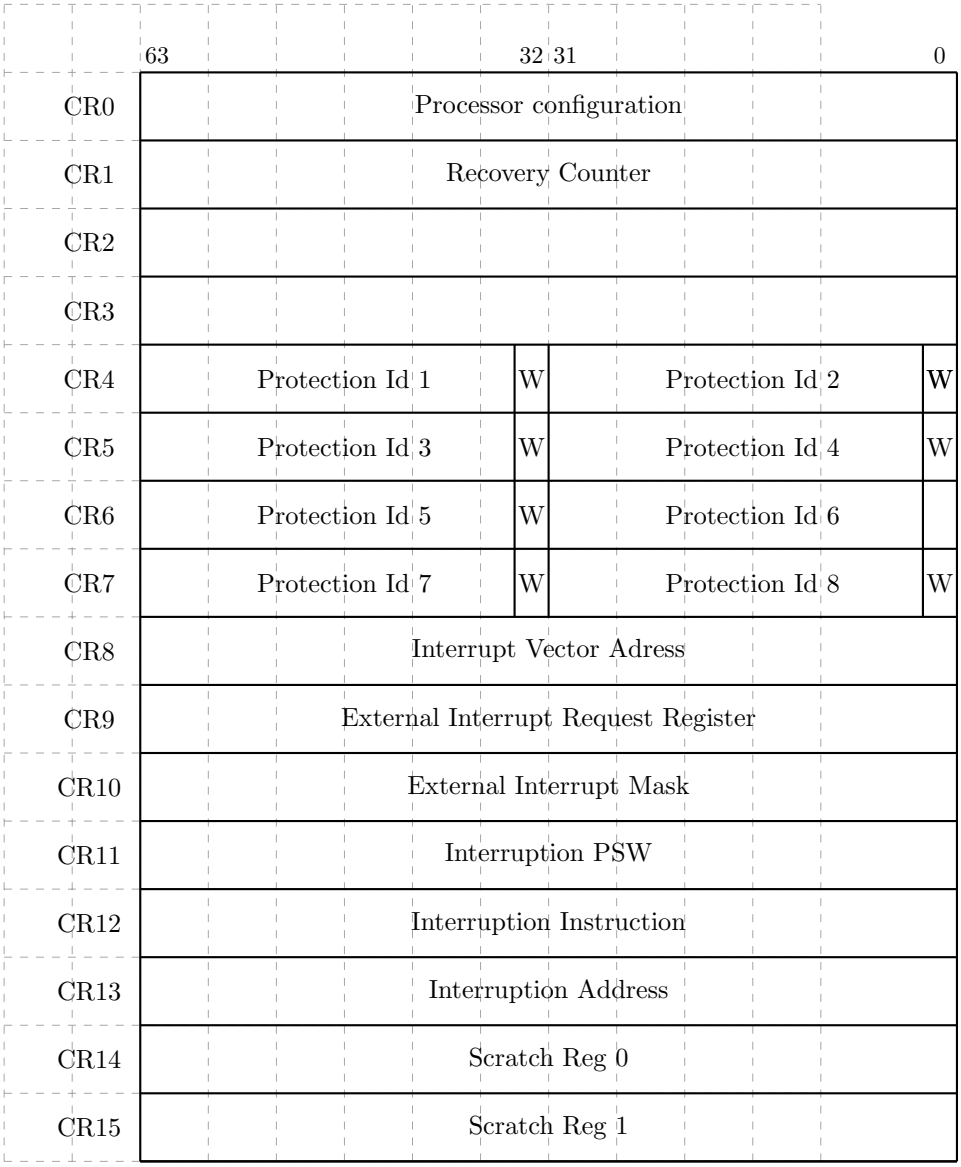


Figure 2: Control registers

Processor Configuration Register

Recovery Counter

Protection Id Registers

contain the segment Id of the segment protected by the "P" bit...

cross check the concept ? we still need the protect Id field in the TLB, but do we need to pass the segment Id in the protect field ? it is encoded in the segment part of the vAdr ...

Interrupt Vector Address

External Interrupt Request Register

??? rather combine EIR and EIM in one register, 32bits each ?

External Interrupt Mask

Interruption PSW

Interruption Instruction

Interruption Address

Scratch Registers

Program State Register

The processor state register contains the virtual address of the current instruction and the processor status flags.

??? finalize status bits...

Figure 3: Program State Register

Instruction Address

Processor Status

M I E and so on ...

Table 1: Status Field Bits

Bit	Purpose
W	When set, a memory reference is in little-endian format.
X	When set, the processor runs in privileged mode.
P	When set, the processor does protection Id checking.
...	...

Data Types

Twin-64 is a big endian 64-bit machine. The fundamental data type is a 64-bit signed integer with the most significant bit being left. All computation is performed on 64-bit signed integer numbers. Memory access is performed on a double, word, half-word or byte basis. Word, Half and Byte data loaded is sign-extended. Store operations just store the corresponding value in memory.

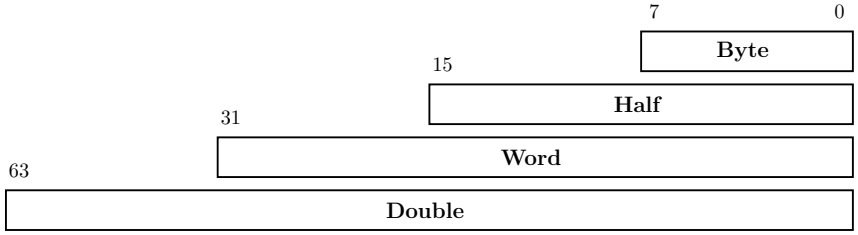


Figure 4: Data types

All memory addresses are expressed as bytes addresses. Address computation uses a 32-bit unsigned math, i.e. numbers wrap around in a 32-bit space.

Byte Ordering

Twin-64 is internally a big-endian machine. It does however support little endian and big endian byte ordering models.

Translation Lookaside Buffer

For performance reasons, the virtual address translation result is stored in a translation lookaside buffer. The TLB is essentially a copy of the page table entry information that represents the virtual page currently loaded at the physical address. Depending on the hardware implementation, there can be a combined TLB or a separate instruction TLB and data TLB. A TLB is essentially a cache for address translations. The virtual page number, VPN, is stored along with attributes and the physical page number, PPN, in the TLB hardware. The current architecture allows for a 34-bit physical address space, which is a 16GB memory space.

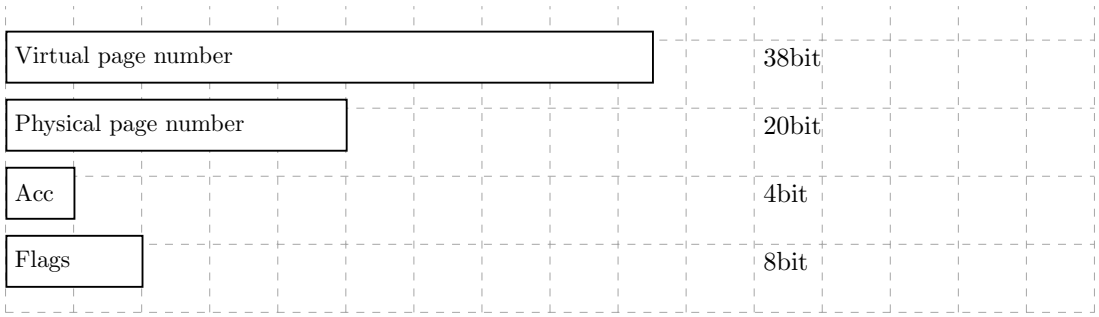


Figure 5: Tlb info

To reduce TLB processing overhead, the TLB is capable of storing virtual address ranges. There is support for a large page size of 16Kb, 256 Kb, 4Mb and 64 Mb.

Table 2: TLB Fields

Field	**Purpose**
VPN	The virtual page number.
PPN	The physical page number.
Access Rights	The access rights of the page.
V	the entry is valid
U	When set, the page represented by the TLB entry is not cached.
P	When set, the page represented by the TLB entry has protection checking enabled.
M	When set, the page represented by the TLB entry has been modified.
B	When set, a branch instruction executed from the page represented by the TLB will cause a branch taken trap.
L	When set, the TLB entry is locked and will not be selected when replacing an entry.
Page Size	The 2-bit field encodes the supported page size. A value of zero indicates a 16Kbyte page, a value of 1 a 256Kbyte, a value of 2 a 4Mbyte and a value of 3 a 64Mbyte page.

The first four segments, segment 0 to 3, are special in that their address translation bypasses the TLB entirely. A virtual address in the range 0x00_0000_0000 to 0x03_FFFF_FFFF directly maps to the physical address of the same range. The address range represents physical memory and can only be accessed in privileged mode.

Caches

Twin-64 is a cache visible architecture, featuring cache models for unified or separate instruction and data cache. While hardware directly controls the cache for cache line insertion and replacement, the software has explicit control on flushing and purging cache line entries. There are instructions to flush and purge cache lines. To speed up the entire memory access process, the cache uses the virtual address to index into the arrays, which can be done in parallel to the TLB lookup. When the physical address tag in the cache and the physical page address in the TLB match, there will be a cache hit. The architecture will not specify the size of cache lines or cache organization. Typically a cache line will hold four to eight machine words, the cache organization is in the simplest case a single set direct mapped cache. The Twin-64 architecture does not specify a particular cache and cache hierarchy model. The architecture just specifies instructions to handle cache operations, such as delete or flushing a cache.

System Memory

Virtual Address Space

Twin-64 features a 52-bit virtual address range. The address range is organized into a 20-bit segment identifier and a 32-bit offset into that segment. The following figure shows the virtual address and how the segment is selected.

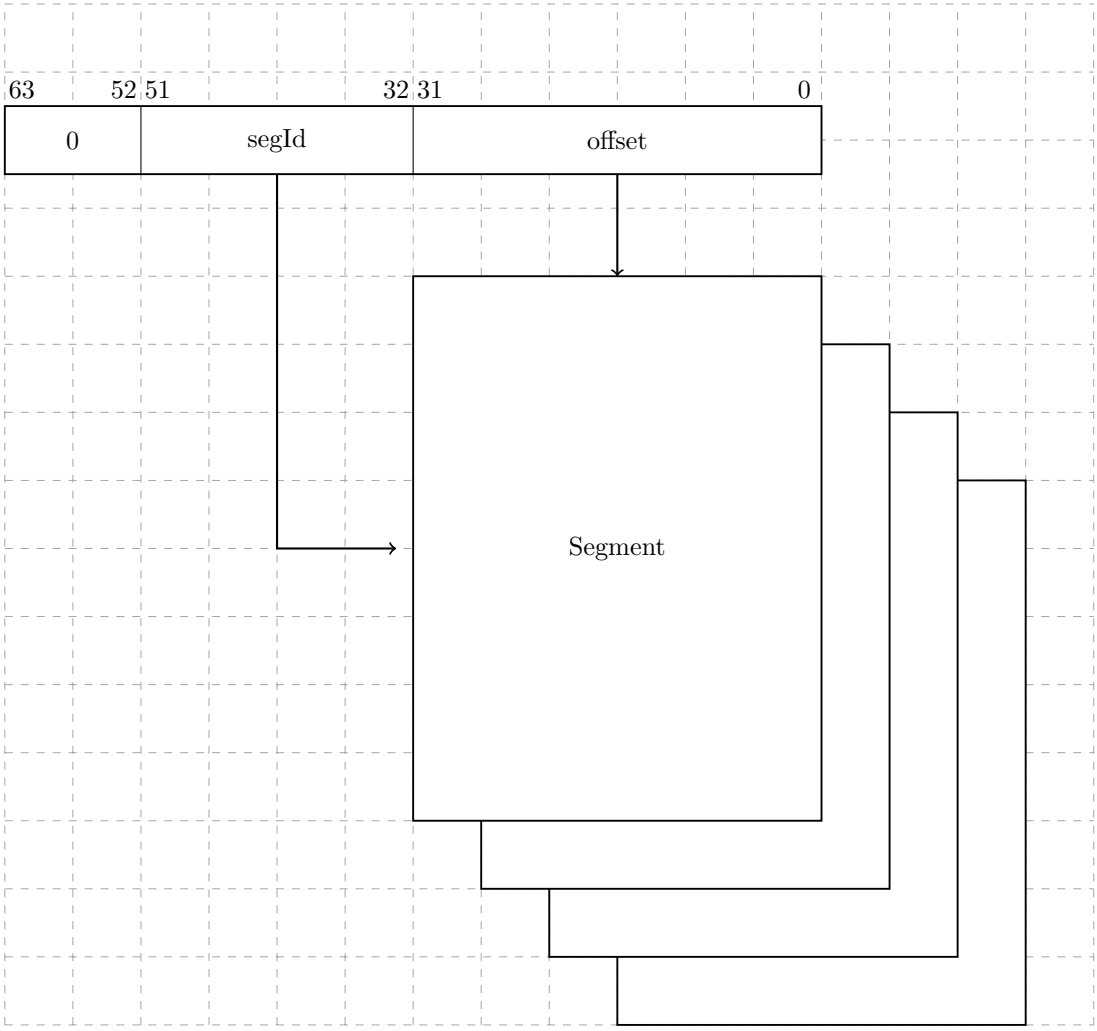


Figure 6: Virtual Memory Address Range

A virtual address to be translated to a physical address. For address translation purposes, the virtual address range can also be seen as a set of virtual pages. The architected page size is 16Kb. A virtual page number is a 38-bit page number with a 14-bit offset into that page.

Physical Address Space

Twin-64 architecture features a 34-bit physical memory address range which allows a maximum physical memory of up to 16 GBytes.

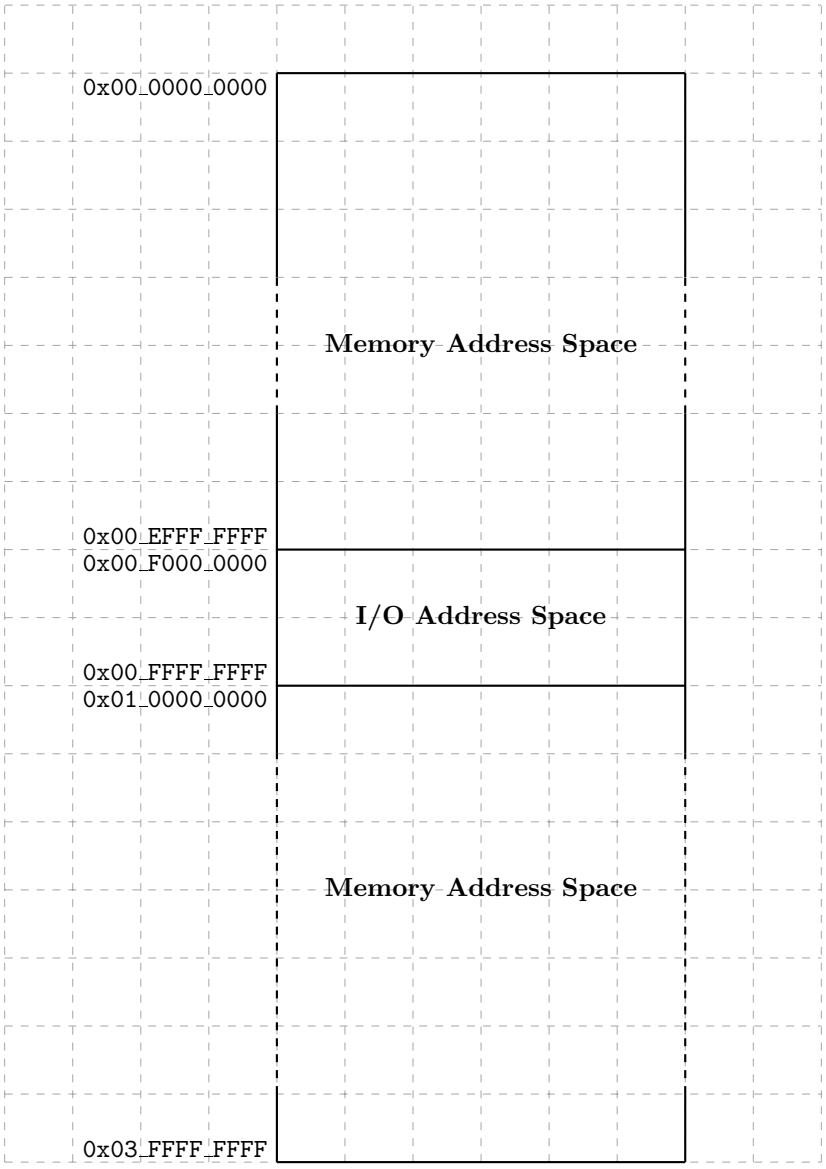


Figure 7: Physical Memory Address Range

segment Id 0 to 3 directly map to the physical address, without TLB translation and access rights checking. Only privileged mode can access these segments.

I/O Address Space

Twin-64 features memory mapped I/O architecture. As shown in the overall physical memory layout, the physical memory address range is located from `0x00_F000_0000` to `0x00_FFFF_FFFF`. The following figure depicts the I/O memory address space ranges in more detail.s

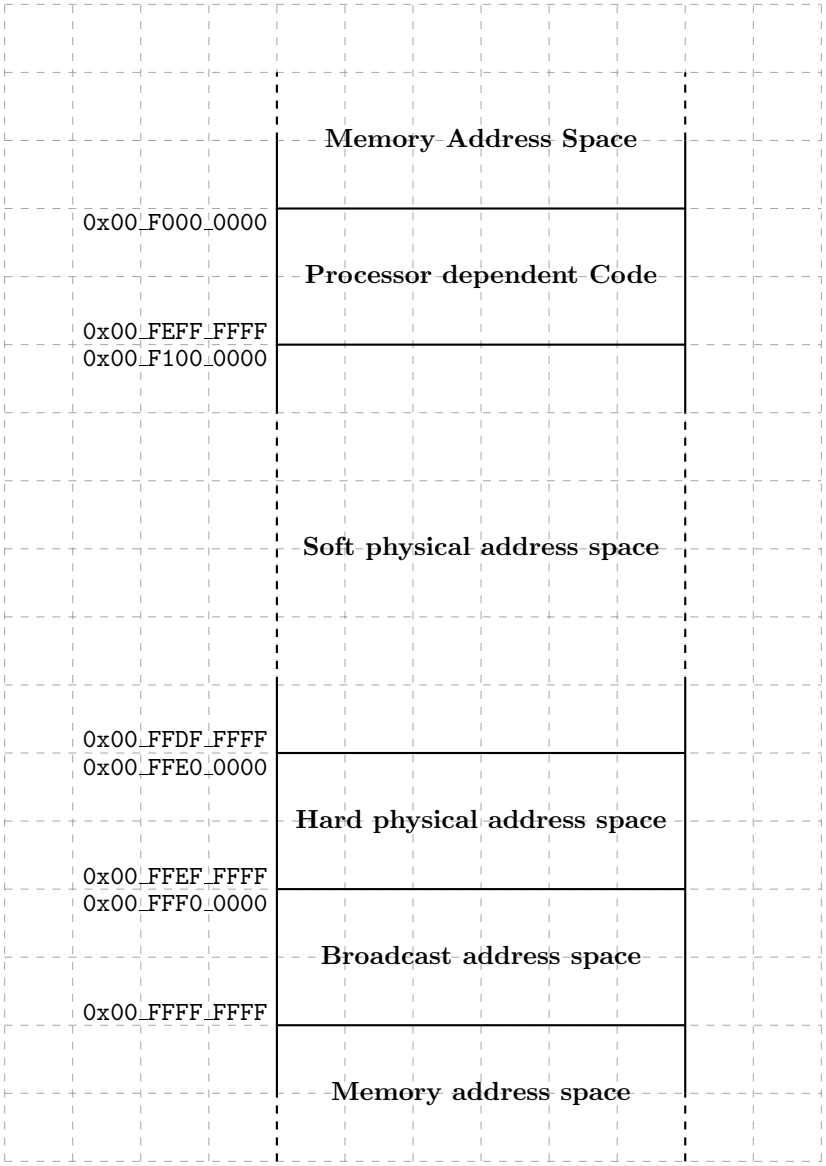


Figure 8: I/O Memory Address Range

always uncached

The I/O address space dedicates 256 Mbytes to an area that contains the processor dependent code. Typically, this is a set of library functions to manage the processor itself but also provide low-level primitives for the boot process and the operating system. The remainder of the I/O address space is divided into I/O modules, which map the I/O hardware components to a memory addressable location. Both PDC and I/O firmware design is explained in a later part of the document.

Addressing and Access Control

The CPU emits a 52-bit virtual addresses. Each virtual memory access needs to be translated and checked for valid access rights.

Address Translation

The Twin-64 CPU emits always a virtual address. Each memory access needs to be translated into a physical address.

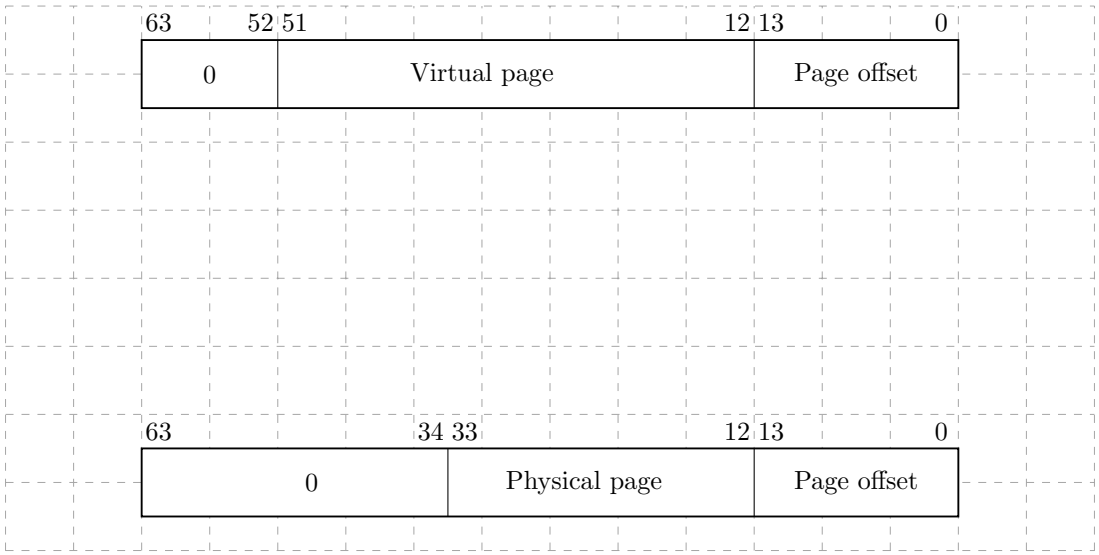


Figure 9: Address translation

virtual address for the first four segments directly map to the physical address space.

Access Control

Twin-64 implements a protection model along two dimensions. The first dimension is the **access control** and **privilege level** check. Each page is associated with a page type. There are read-only, read-write, execute and gateway pages. Each memory access is checked to be compatible with the allowed type of access set in the page descriptor. There are two privilege levels, user and supervisor mode. Access type and privilege level form the access control information field, which is checked for each instruction and data memory access. For read access the privilege level us be least as high as the PL1 field, for write access the privilege level must be as least as high as PL2. An exception are the execute and gateway pages, where a write access is only allowed for privilege code. For execute access the privilege level must be at least as high as PL1 and not higher than PL2. If the instruction privilege level is not within this bounds, a privilege

violation trap is raised. If the page type does not match the instruction access type an access violation trap is raised. In both cases the instruction is aborted and a memory protection trap is raised.

Access Protection

The second dimension of protection is a **protection ID**. Twin-64 allows to record a set of segment Id values in the processor control registers. For each access to a segment that has the protection check bit set, one of the protection Id control registers must match the segment Id. If not, a protection violation trap is raised.

Protection Id checking is typically used to form a grouping of access. A good example is the stack data segment, which is accessible at user privilege level. Since the CPU features a global virtual memory address space, every task could access a data stack of another task. With protection Id checking enabled and the segment Id loaded in one of the control registers, only the corresponding user task is allowed to access the stack data segment with a matching protection Id.

Interruptions

Traps

Table 3: Traps

Head 1	Head 2
text 1	text
text 2	text

External Interrupts

Input/Output

Concepts

memory mapped

dedicated address space in physical memory

bus concept

module concept

modules have registers - load / store, however registers do not have to be 64 bit wide.

the system bus features up to 64 modules. that is: 64 HPA pages, need: 1Mb address range.

a module listens to three address ranges

- hard physical address range. a page. privileged.

- soft physical address range. allocated by the module for further space. aligned on a page boundary.

- broadcast address range. Mirrors the system bus range. A write to a register in that range applies to all corresponding registers of a module on the same bus.

modules are processors, memory and I/O cards.

we could keep processor stats in the HPA space

memory module has entire memory as SPA space

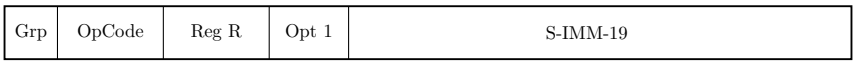
Instruction Set

Instruction Overview

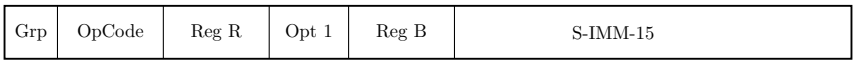
This chapter presents the Twin-64 instruction set. Like typical RISC style instruction sets, it is a fixed length instruction set with very few regular formats. The instructions are group into four categories, arithmetic and logical, load and store, branch and special and system instructions.

Instruction Formats

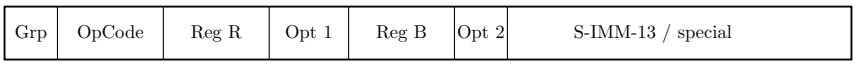
Twin-64 uses few instruction formats. They are grouped by the number of available register slots and the length of the immediate value field. The **signed immediate S19** instruction format provides one register and a 19 bit signed immediate field. This format is typically used by branch instructions.



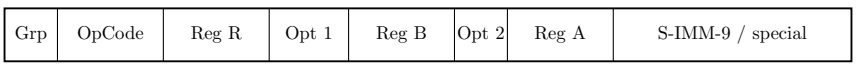
The **signed immediate S15** instruction format provides two registers and a 15 bit signed immediate field. This format is used by branch instructions as well as instruction which operate on a register and an immediate value.



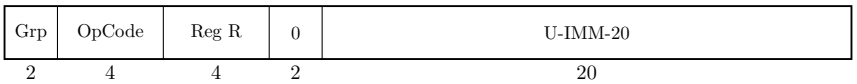
The **signed immediate S13/special** instruction format provides two registers, an additional option field and a 13 bit signed immediate field. Some instruction use the S-IMM-13 field for special encodings instead of a signed value. This format is used by memory reference instructions where the address is formed by a base register and a 13-bit signed offset.



The **register/signed immediate S9/special** instruction format provides three registers, an additional option field and a 9 bit signed immediate field. Some instruction use the S-IMM-9 field for special encodings instead of a signed value.instead of a signed value. This instruction format is used for all computational instructions where three registers are needed.



The **unsigned immediate U20** format is used by the immediate instruction group to provide a 20bit unsigned value. This value is then placed at certain positions in the register target Reg R.



Arithmetic Instructions

The arithmetic instruction operate on two signed 64-bit operands and stores the result to the target register. There are two instruction layouts, signed immediate S15 and register based. The **immediate operand instruction format** will add the sign extended 15-bit value to `RegB`. The **register instruction format**type instruction uses two registers as input operands, performs the operation and stores the result in a third register. The following figure shows the instruction layout.

The **ADD** and **SUB** instructions perform signed integer 64-bit arithmetic with a numeric overflow resulting in an overflow trap. There are no operations on unsigned quantities. The **SHLxA** and **SHRxA** perform an arithmetic left shift operation and add an operand to the shifted input, storing the result in the target register.

The **CMP** compares two registers for being equal, not equal, less than and les or equal than. The result of this comparison is store din the target register.

Bit Operations Instructions

The bit operation instruction fall into two groups. The **AND**, **OR** and **XOR** instructions implement the logical operations as their name suggest. Like the arithmetic instructions, there is an immediate instruction and a register instruction layout.

The second group implements the bit manipulation operations. The **EXTR** instruction extracts a bit field from a register and places the field in another register. Optionally, the extracted field is signed extended. The **DEP** instruction deposits a bit field into a target register.

The **DSR** instruction concatenates two general registers, shifts the 128-bit quantity to the right and stores the lower 64 bits in a target register.

Immediate Instructions

The **LDI**, and **ADDIL** are instruction for forming large constants. With a 32-bit instruction word, even 32-bit constants are not possible in one instruction. The **LDI** instruction has a qualifier to load a constant value at a defined position in the target register. The opt field allow to select fields in the target register where the constant value is stored. The **ADDIL** instruction adds the upper portion of a 32-bit immediate value to the lower 32-bit half of a general register. This instruction is used primarily for address arithmetic to allow reaching any location in a segment.

The **LDO** instruction loads an offset into a general register. The address loaded is formed by adding the signed immediate value to the base register.

Memory Reference Instructions

Twin-64 is a register memory architecture. It has the common load/store instructions as well as instructions that combined an operation with a memory reference. Memory reference instructions feature two addressing modes. The first is the base register and offset instruction. the second the base register and register index instruction.

The **indexed instruction format** will load the content of memory address formed by adding the scaled immediate 13-bit field to the base register `RegB` and add the data value to `RegA`. The **dw** field indicates the data value width. A **dw** value of zero load an 8-bit, a value of 1 a 16-bit value, value of 2 a 32-bit value and a value of 3 a 64-bit value. In any case, the data value fetched is signed extended to a 64-bit value. In addition, the **dw** field will scale the offset value by left shifting the immediate field according to the data width. The following figure shows the general instruction layout for base register and offset instruction.

The **register indexed instruction format** will load the content of memory address formed by adding `RegX` to the base register `RegB` and add the data value to `RegA`. Both load formats will use the **dw** field to specify the data value width. A **dw** value of zero load an 8-bit, a value of 1 a 16-bit value, value of 2 a 32-bit value and a value of 3 a 64-bit value. In any case, the data value fetched is signed extended to a 64-bit value. The offset in `RegX` is interpreted as a byte offset, independent of the actual **dw** field value. The following figure shows the general instruction layout for base register and register index type instructions.

The **LD** and **ST** instruction are the load and store instructions. In addition to the two addressing modes presented above, they also feature a base register address modification. Depending on the offset sign, the base register is updated before or after the address computation with the effective address computed. See the instruction description for more details.

The **LDR** and **STC** instruction implement the base support for a pipeline friendly method for semaphore operations. They only support the base register and offset mode. Also, there is no base register modification option.

The **ADD**, **SUB**, **AND**, **OR**, **XOR**, **CMP** instructions perform the respective operation as described above with one operand being fetched from memory.

Branch Instructions

Control flow is implemented through a set of branch instructions. They can be classified into unconditional and conditional instruction branches.

Unconditional branches fetch the next instruction address from the computed branch target. The **B** instruction performs an instruction relative branch, the **BR** instruction a base register relative branch. Both optionally return the return address in a general register.

The **BV** instruction is a vectored branch. The content of a general register is added to a base register, forming branch target.

The instructions **BB**, **CBR** and **MBR** implement the conditional branch instructions. If the condition is met a branch to the target address is performed. The target address is always a local address and the offset is instruction address relative. Conditional branches adopt a static prediction model. Forward branches are assumed not taken, backward branches are assumed taken.

All branch address arithmetic is a 32-bit unsigned arithmetic with wraparound. Adding branch offsets will not overflow into the segment Id portion of the address..

System Instructions

The final instruction group is the **special/system** instruction group. The system control instructions are intended for the runtime designer to control the CPU resources such as TLB, Cache and so on. There are instruction to load a segment or control registers as well as instructions to store them. The TLB and the cache modules are controlled by instructions to insert and remove data in these two entities. Finally, the interrupt and exception system needs a place to store the address and instruction that was interrupted as well as a set of shadow registers to start processing an interrupt or exception. Most of the instructions found in this group require that the processor runs in privileged mode.

The **MFCR**and **MTCR** instructions are used to transfer data to and from a control register. Access to some control registers requires privileged mode. The **RSM**and **SSM** instructions allow for setting or clearing an individual an accessible bit of the status part of the processor state.

The **LDPA**, **PRBR** and **PRBW** instructions are used to obtain information about the virtual memory system. The LDPA instruction returns the physical address of the virtual page, if present in memory. The PRB instruction tests whether the desired access mode to an address is allowed.

The TLB and Caches are managed by the **ITLB**, **PTLB**, **FCA** and **PCA** instruction. The ITLB and PTLB instructions insert into the TLB and remove translations from the TLB. The FCA and PCA instructions manages the instruction and data cache, featuring to flush and / or just purge a cache line. There is no instruction that inserts data into a cache as caching is completely controlled by hardware.

The **RFI** instruction is used to restore a processor state from the control registers holding the interrupted instruction address, the instruction itself and the processor status word. Optionally, general registers that have been stored into the reserved control registers will be restored.

The **DIAG** instruction is a control instructions to issue hardware specific implementation commands. Example is the memory barrier command, which ensures that all memory access operations are completed after the execution of this DIAG instruction. The **BRK** instruction is transferring control to the breakpoint trap handler.

Instruction descriptions

The instructions described in this chapter contains the instruction word layout, a short description of the instruction and also a high level pseudo code of what the instruction does. Each

instruction operation is described in a C-style pseudo code operation. The pseudo code uses a register notation describing by their class and index. For example, GR[5] labels general register 5, SR[2] represents the segment register number 2 and CR[1] the control register number 1.

In addition, some register also have dedicated names, such as for example the shift amount control register, labelled "shamt". A subfield of the instruction is selected by adding a "." and the field name in brackets. For example the carry bit in the PSW register is described as "PSW.[C]". The pseudo code routines used in the instruction descriptions are listed in the appendix.

Additional options for an instruction will be specified by a "." followed by a sequence of characters. Each character is one of the defined options for the instruction. As an example, the AND instruction can negate the result. The assembler notation would be "AND.N ...". The order of the individual characters does not matter. The defined options are listed as a list of characters.

Reserved fields in an instruction word are future enhancements and should be filled with a zero. The instruction explicitly shows the numeric value of these fields. Any other value at that place will result in an undefined behavior.

ADD Integer Addition

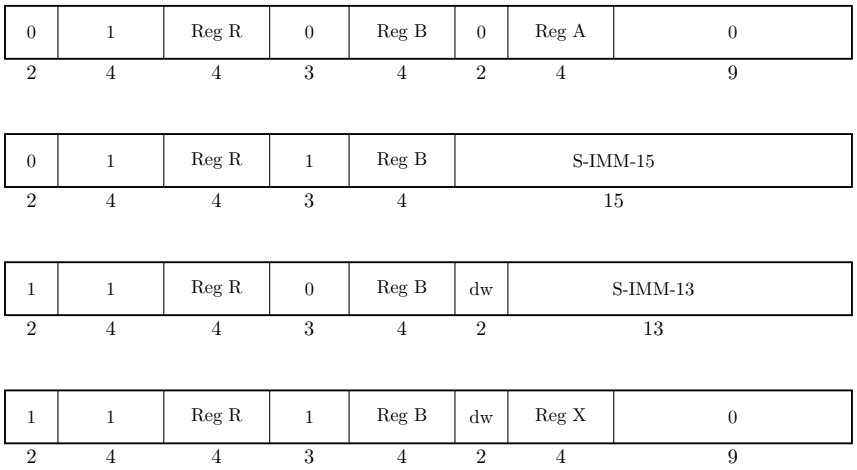
Syntax

```
ADD RegR, RegB, RegA
ADD RegR, RegB, SignedImmed15

ADD[.D/W/H/B] RegR, SignedImmed13( RegB )
ADD.D/W/H/B] RegR, RegX ( RegB )
```

Format

The ADD instruction uses the register, the immediate, the indexed and the register indexed instruction formats.



Description

The `ADD` instruction adds two signed 64-bit operands and stores them to the target register `RegR`. The instruction supports the immediate, the register and the two memory reference instruction formats. An arithmetic overflow results in a numeric overflow trap. The indexed formats may cause a memory reference associated trap.

Operation

`RegR <- RegB + RegA` (register format)
`RegR <- RegB + immOperand( )` (immediate format)
`RegR <- RegR + memOperand( )` (indexed formats)

Exceptions

Integer Overflow Trap
Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

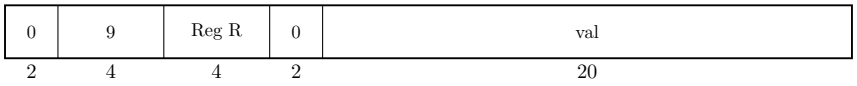
Notes

None.

ADDIL Add immediate left

Syntax `ADDIL RegR, val`

Format The ADDIL instruction uses the immediate-20 instruction format.



Description The `ADDIL` instruction ...

Operation

```
RegR <- RegR + ( val << 12 );
```

Exceptions None.

Notes None.

AND Boolean Operation

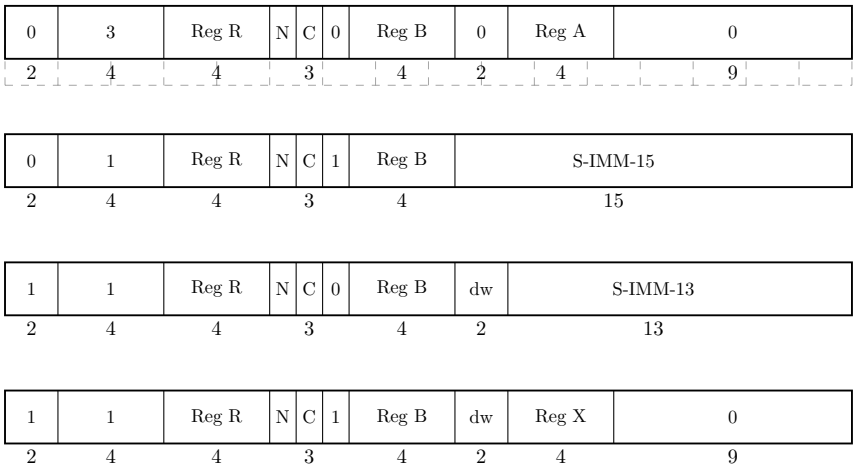
Syntax

```
AND[.C/N] RegR, RegB, RegA
AND[.C/N] RegR, RegB, SignedImmed15

AND[.C/N/D/W/H/B] RegR, SignedImmed13( RegB )
AND[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The AND instruction uses the register, the immediate, the indexed and the register indexed instruction formats.



Description

The `AND` instruction performs the binary AND operation on two 64-bit operands and stores the result to the target register `RegR`. Upon input the general register `regB` can be negated. The result can also be negated by setting the "C" bit. The result can be negated by setting the "N" bit. The instruction supports the immediate, the register and the two memory reference instruction formats. The boolean operation itself does not cause any traps.The indexed formats may cause a memory reference associated trap.

Operation

```
if ( instr.[C] RegB <- not( RegB );
RegR <- RegB & RegA (register format)
RegR <- RegB & immOperand( ) (immediate format)
RegR <- RegR & memOperand( ) (indexed formats)
if ( instr.[N] RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

CMP Integer Compare

Syntax

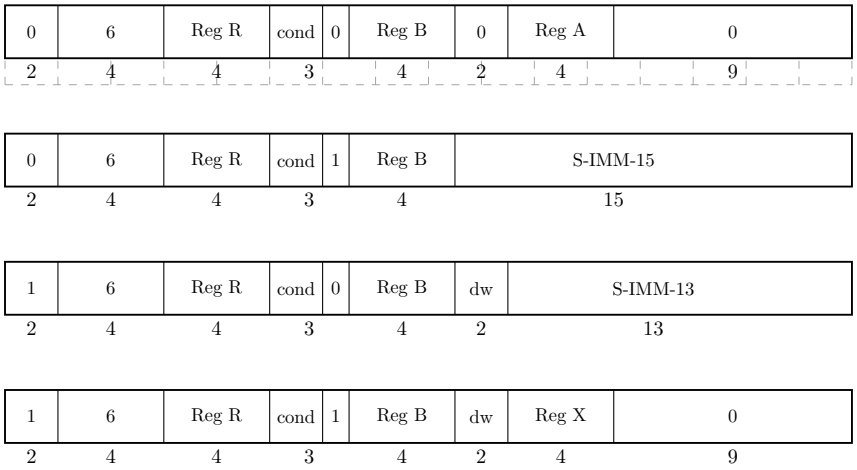
```
CMP.<cond> RegR, RegB, RegA
CMP.<cond> RegR, RegB, SignedImmed15

CMP.<cond>[.C/N/D/W/H/B] RegR, SignedImmed13( RegB )
CMP.<cond>[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

where ¡cond¿ is EQ, LT, NE or LE.

Format

The CMP instruction uses the register, the immediate, the indexed and the register indexed instruction formats.



Description

The `CMP` instruction performs an integer comparison of the two 64bit signed operands and depending on the comparison stores a 0 or 1 in the target register. The indexed formats may cause a memory reference associated trap.

The condition field encodes the comparison options. The options are a 0 for equal, a 1 for less than, a 2 for not equal and a 3 for less than or equal.

Operation

```
cond <- instr.[cond];
RegR <- cmpOp( cond, RegB, RegA ) (register format)
RegR <- cmpOp( cond, RegB, immOperand( )) (immediate format)
RegR <- cmpOp( cond, RegR, memOperand( )) (indexed formats)
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

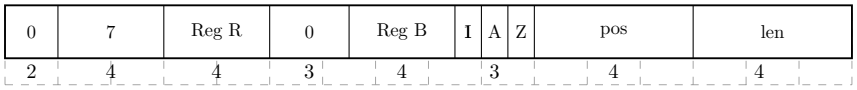
DEP Deposit Bit Field

Syntax

```
DEP[.Z/I] RegR, RegB, pos, len
DEP[.Z/I] RegR, RegB, SAR, len
```

Format

The DEP instruction instruction uses the register instruction format.



Description

The DEP instruction deposits the bit field of length "len" in general register RegB into the general register RegR at the specified position. The position field specifies the rightmost bit for the bit field to deposit. The length field specifies the bit size of the field to deposit. The Z bit clears the target register before storing the bit field. If the A bit is set, the shift amount control register is used for obtaining the position value. The I bit specifies that the RegA field contains an immediate value instead of a register.

Operation

```
if ( instr.[Z] ) RegR <- 0;
if ( instr.[A] ) pos <- shamt;
else pos <- instr.[pos];
if ( instr.[I] )
RegR <- depositImm( RegR, RegB, pos, instr.[6:0] );
else
RegR <- depositField( RegR, RegB, pos, instr.[6:0] );
```

Exceptions

None

Notes

None.

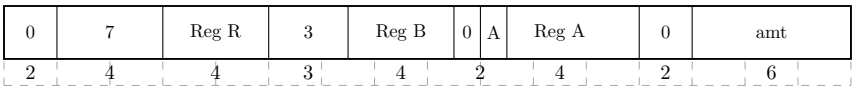
DSR Double Shift Right

Syntax

```
DSR RegR, RegB, RegA, amt
DSR RegR, RegB, RegA, SAR
```

Format

The DSR instruction uses the register instruction format.



Description

The `DSR` instruction concatenates the general registers RegB and RegA and performs a right shift operation. Register RegB is the left part, register RegA the right part. The lower 64 bits of the result are stored in the general register RegR. The amount field contains the shift amount. If the A bit is set, the shift amount is taken from the shift amount control register.

Operation

```
if ( instr.[A] ) tmp <- shamtReg;
else tmp <- instr.[amt];
RegR <- doubleShiftR( RegB, RegA, tmp );
```

Exceptions

None.

Notes

None.

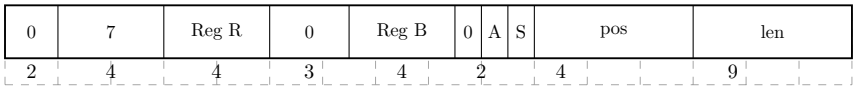
EXTR Extract Bit Field

Syntax

```
EXTR[.S] RegR, RegB, pos, len
EXTR[.S] RegR, RegB, SAR, len
```

Format

The EXTR instruction uses the immediate-20 instruction format.



Description

The EXTR instruction performs a bit field extract specified by the position and length instruction field from general register RegB. The position field specifies the rightmost bit of the bitfield to extract. The length field specifies the bit size if the field to extract. The extracted bit field is stored right justified in the general register RegR. If the S bit is set, the result is sign extended. If the A bit is set, the shift amount control register is used for obtaining the position value instead of the length instruction field.

Operation

```
if ( instr.[A] tmp <- shamt;
else tmp <- instr.[pos];
RegR <- extractField( RegB, pos, len );
if ( instr.[S] ) RegR <- signExtend( RegR, pos );
```

Exceptions

None

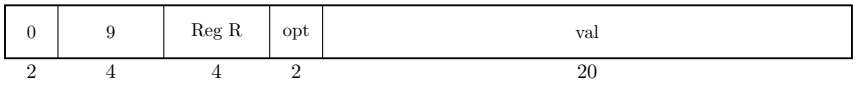
Notes

None.

LDI Load immediate

Syntax `LDI[.L/S/U] RegR, val`

Format The LDI instruction uses the immediate-20 instruction format.



Description The `LDI` instruction ...

Operation

```
RegR <- (( val & 0xFFFFF ) << 12 );  (.L option )
RegR <- (( val & 0xFFFFF ) << 32 );  (.S option )
RegR <- (( val & 0xFFF ) << 52 );  (.U option )
```

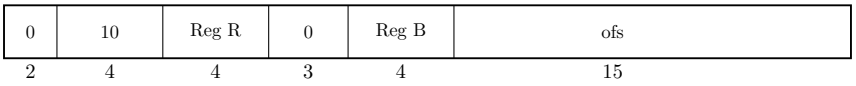
Exceptions None.

Notes None.

LDO load offset

Syntax `LDO RegR, ofs ( RegB )`

Format The LDO instruction uses the indexed instruction format.



Description The `LDO` instruction ...

Operation `RegR <- RegR + ofs;`

Exceptions None.

Notes None.

OR Boolean Operation

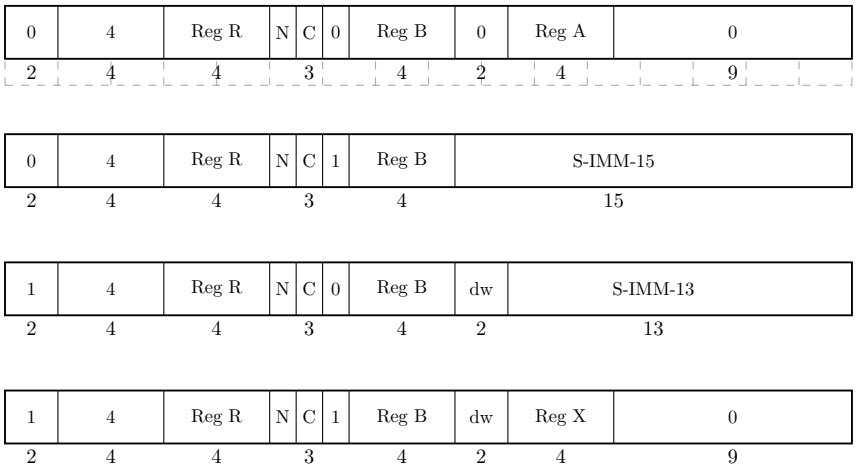
Syntax

```
OR[.C/N] RegR, RegB, RegA
OR[.C/N] RegR, RegB, SignedImmed15

OR[.C/N/D/W/H/B] RegR, SignedImmed13( RegB )
OR[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The OR instruction uses the register, the immediate, the indexed and the register indexed instruction formats.



Description

The `OR` instruction performs the binary OR operation on two 64-bit operands and stores the result to the target register `RegR`. Upon input the general register `regB` can be negated. The result can also be negated by setting the "C" bit. The result can be negated by setting the "N" bit. The instruction supports the immediate, the register and the two memory reference instruction formats. The boolean operation itself does not cause any traps.The indexed formats may cause a memory reference associated trap.

Operation

```
if ( instr.[C] RegB <- not( RegB );
RegR <- RegB | RegA (register format)
RegR <- RegB | immOperand( ) (immediate format)
RegR <- RegR | memOperand( ) (indexed formats)
if ( instr.[N] RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

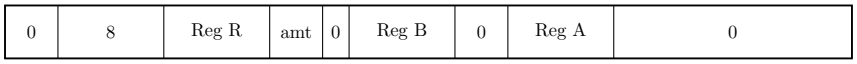
SHLxA Shift left and add

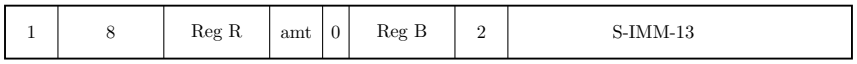
Syntax

```
SHLxA RegR, RegB, RegA
SHLxA RegR, RegB, SignedImmed13
```

Format

The SHLxA instruction uses the register and the immediate instruction formats.





Description

The `SHLxA` instruction ...

Operation

`RegR <-( RegB << amt ) + RegA` (register format)
`RegR <- ( RegB << amt ) + immOperand( )` (immediate format)

Exceptions

Integer Overflow Trap

Notes

None.

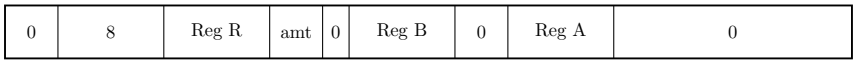
SHRxA Shift right and add

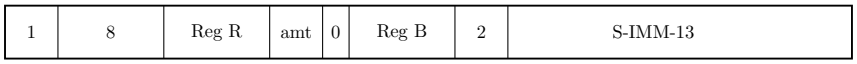
Syntax

```
SHRxA RegR, RegB, RegA
SHRxA RegR, RegB, SignedImmed13
```

Format

The SHRxA instruction uses the register and immediate instruction formats.





Description

The `SHRxA` instruction ...

Operation

`RegR <-( RegB >> amt ) + RegA` (register format)
`RegR <- ( RegB >> amt ) + immOperand( )` (immediate format)

Exceptions

Integer Overflow Trap

Notes

None.

SUB Integer Subtraction

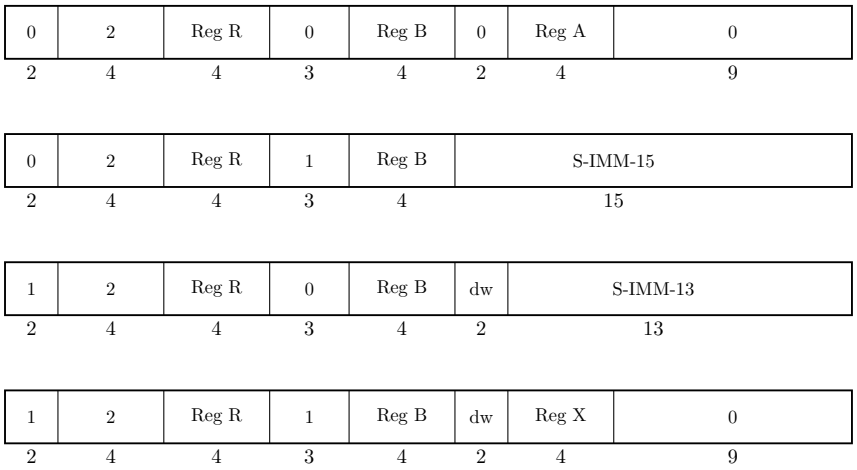
Syntax

```
SUB RegR, RegB, RegA
SUB RegR, RegB, SignedImmed15

SUB[.D/W/H/B] RegR, SignedImmed13( RegB )
SUB.D/W/H/B] RegR, RegX ( RegB )
```

Format

The SUB instruction uses the register, the immediate, the indexed and the register indexed instruction formats.



Description

The `SUB` instruction subtracts a signed 64-bit operand from a general register `RegB` and stores the result to the target register `RegR`. The instruction supports the immediate, the register and the two memory reference instruction formats. An arithmetic overflow results in a numeric overflow trap. The indexed formats may cause a memory reference associated trap.

Operation

`RegR <- RegB - RegA` (register format)
`RegR <- RegB - immOperand( )` (immediate format)
`RegR <- RegR - memOperand( )` (indexed formats)

Exceptions

Integer Overflow Trap
Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

XOR Boolean Operation

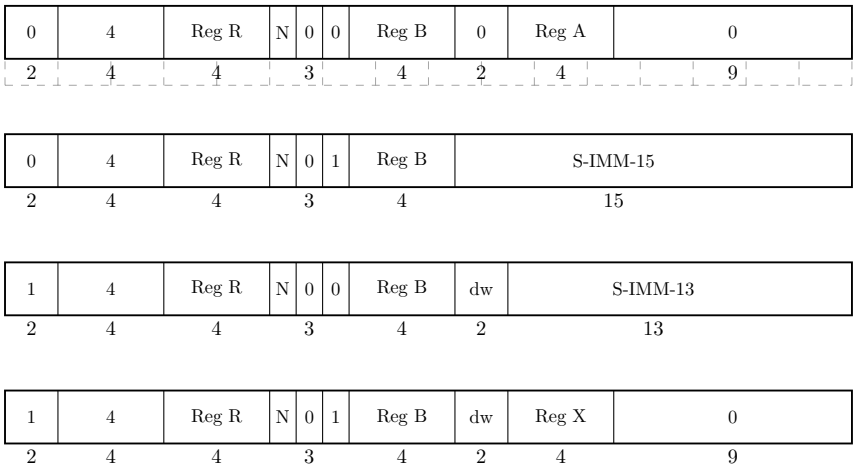
Syntax

```
XOR[.N] RegR, RegB, RegA
XOR[.N] RegR, RegB, SignedImmed15

OR[.N/D/W/H/B] RegR, SignedImmed13( RegB )
OR[.N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The XOR instruction uses the register, the immediate, the indexed and the register indexed instruction formats.



Description

The `XOR` instruction performs the binary XOR operation on two 64-bit operands and stores the result to the target register `RegR`. The result can be negated by setting the "N" bit. The instruction supports the immediate, the register and the two memory reference instruction formats. The boolean operation itself does not cause any traps.The indexed formats may cause a memory reference associated trap.

Operation

`RegR <- RegB ^ RegA` (register format)
`RegR <- RegB ^ immOperand( )` (immediate format)
`RegR <- RegR ^ memOperand( )` (indexed formats)
`if ( instr.[N] RegR <- not( RegR );`

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.